

2. Advanced publish

Nextflow from zero to hero

2026/01/28

Output files in a pipeline

Work vs Publish

- Files created by a process stay in the working directory...
- ... but we can *publish* them to another location using the `publishDir` directive
 - rename
 - decide how to publish
 - control which files are published
 - organise in logical directory structures

publishDir

```
process PROCESS_SAMPLE {  
  
    publishDir path: "results"  
  
    input:  
    tuple val(sample_id), val(species)  
  
    output:  
    path "*.fasta"  
    path "*.csv"  
    path "*.bed"  
  
    script:  
    """  
    echo "here a FASTA file" > ${sample_id}_${species}.fasta  
    echo "here a CSV file" > ${sample_id}_${species}.csv  
    echo "here a BED file" > ${sample_id}_${species}.bed  
    """  
}
```

Preview DAG

```
workflow {  
  
    samples_species_ch = channel.of(  
        ['sampleA', 'mice'],  
        ['sampleB', 'rat']  
    )  
    PROCESS_SAMPLE(samples_species_ch)  
}
```

publishDir path;

```
process PROCESS_SAMPLE {  
  
    publishDir path: "results"  
  
    input:  
    tuple val(sample_id), val(species)  
  
    output:  
    path "*.fasta"  
    path "*.csv"  
    path "*.bed"  
  
    script:  
    """  
    echo "here a FASTA file" > ${sample_id}_${species}.fasta  
    echo "here a CSV file" > ${sample_id}_${species}.csv  
    echo "here a BED file" > ${sample_id}_${species}.bed  
    """  
}
```

Preview DAG

```
workflow {  
  
    samples_species_ch = channel.of(  
        ['sampleA', 'mice'],  
        ['sampleB', 'rat']  
    )  
  
    PROCESS_SAMPLE(samples_species_ch)  
}
```

Only files that **match** the declaration in the **output** block are published, not all the outputs of the process.

```
.  
├── work/  
│   ├── sampleA_mice.fasta  
│   ├── sampleA_mice.csv  
│   ├── sampleA_mice.bed  
│   ├── sampleB_rat.fasta  
│   ├── sampleB_rat.csv  
│   └── sampleB_rat.bed  
└── results/  
    ├── sampleA_mice.fasta  
    ├── sampleA_mice.csv  
    ├── sampleA_mice.bed  
    ├── sampleB_rat.fasta  
    ├── sampleB_rat.csv  
    └── sampleB_rat.bed
```

publishDir path;

```
process PROCESS_SAMPLE {  
  
    publishDir path: "results"  
  
    input:  
    tuple val(sample_id), val(species)  
  
    output:  
    path "sampleA*.fasta"  
    path "sampleA*.csv"  
    path "sampleA*.bed"  
  
    script:  
    """  
    echo "here a FASTA file" > ${sample_id}_${species}.fasta  
    echo "here a CSV file" > ${sample_id}_${species}.csv  
    echo "here a BED file" > ${sample_id}_${species}.bed  
    """  
}
```

Preview DAG

```
workflow {  
  
    samples_species_ch = channel.of(  
        ['sampleA', 'mice'],  
        ['sampleB', 'rat']  
    )  
  
    PROCESS_SAMPLE(samples_species_ch)  
}
```

Only files that **match** the declaration in the **output** block are published, not all the outputs of the process.

```
.  
├─ work/  
│   ├── sampleA_mice.fasta  
│   ├── sampleA_mice.csv  
│   ├── sampleA_mice.bed  
│   ├── sampleB_rat.fasta  
│   ├── sampleB_rat.csv  
│   └─ sampleB_rat.bed  
└─ results/  
    ├── sampleA_mice.fasta  
    ├── sampleA_mice.csv  
    └─ sampleA_mice.bed
```

publishDir path; mode;

```
process PROCESS_SAMPLE {  
    publishDir path: "results/${sample_id}"  
  
    input:  
    tuple val(sample_id), val(species)  
  
    output:  
    path "*.fasta"  
    path "*.csv"  
    path "*.bed"  
  
    script:  
    """  
    echo "here a FASTA file" > ${sample_id}_${species}.fasta  
    echo "here a CSV file" > ${sample_id}_${species}.csv  
    echo "here a BED file" > ${sample_id}_${species}.bed  
    """  
}  
  
workflow {  
    samples_species_ch = channel.of(  
        ['sampleA', 'mice'],  
        ['sampleB', 'rat']  
    )  
    PROCESS_SAMPLE(samples_species_ch)  
}
```

By default, files are published with mode: 'symlink'

```
.  
├─ work/  
│   ├── sampleA_mice.fasta  
│   ├── sampleA_mice.csv  
│   ├── sampleA_mice.bed  
│   ├── sampleB_rat.fasta  
│   ├── sampleB_rat.csv  
│   └─ sampleB_rat.bed  
└─ results/  
    ├── sampleA/  
    │   ├── sampleA_mice.fasta -> /abs/path/to/work/sampleA_mice.fasta  
    │   ├── sampleA_mice.csv   -> /abs/path/to/work/sampleA_mice.csv  
    │   └─ sampleA_mice.bed    -> /abs/path/to/work/sampleA_mice.bed  
    └─ sampleB/  
        ├── sampleB_rat.fasta -> /abs/path/to/work/sampleB_rat.fasta  
        ├── sampleB_rat.csv   -> /abs/path/to/work/sampleB_rat.csv  
        └─ sampleB_rat.bed    -> /abs/path/to/work/sampleB_rat.bed
```

publishDir path; mode;

mode

The file publishing method. Can be one of the following values:

- `'copy'`: Copies the output files into the publish directory.
- `'copyNoFollow'`: Copies the output files into the publish directory without following symlinks ie. copies the links themselves.
- `'link'`: Creates a hard link in the publish directory for each output file.
- `'move'`: Moves the output files into the publish directory. **Note:** this is only supposed to be used for a *terminal* process i.e. a process whose output is not consumed by any other downstream process.
- `'rellink'`: Creates a relative symbolic link in the publish directory for each output file.
- `'symlink'`: Creates an absolute symbolic link in the publish directory for each output file (default).

publishDir path; mode; pattern;

```
process PROCESS_SAMPLE {  
  
  publishDir path: "results/${sample_id}/fasta", pattern: "*.fasta"  
  publishDir path: "results/${sample_id}/csv", pattern: "*.csv"  
  publishDir path: "results/${sample_id}/bed", pattern: "*.bed"  
  
  input:  
  tuple val(sample_id), val(species)  
  
  output:  
  path "*.fasta"  
  path "*.csv"  
  path "*.bed"  
  
  script:  
  """  
  echo "here a FASTA file" > ${sample_id}_${species}.fasta  
  echo "here a CSV file" > ${sample_id}_${species}.csv  
  echo "here a BED file" > ${sample_id}_${species}.bed  
  """  
}
```

Preview DAG

```
workflow {  
  
  samples_species_ch = channel.of(  
    ['sampleA', 'mice'],  
    ['sampleB', 'rat']  
  )  
  
  PROCESS_SAMPLE(samples_species_ch)  
}
```

- **publishDir** can be specified more than once in order to publish output files to different target directories
- **pattern** selects which files to publish from the overall set of output files.

```
.  
├─ work/  
│   ├── sampleA_mice.fasta  
│   ├── sampleA_mice.csv  
│   ├── sampleA_mice.bed  
│   ├── sampleB_rat.fasta  
│   ├── sampleB_rat.csv  
│   └── sampleB_rat.bed  
└─ results/  
    ├── sampleA/  
    │   ├── fasta/  
    │   │   └── sampleA_mice.fasta  
    │   ├── csv/  
    │   │   └── sampleA_mice.csv  
    │   └── bed/  
    │       └── sampleA_mice.bed  
    └── sampleB/  
        ├── fasta/  
        │   └── sampleB_rat.fasta  
        ├── csv/  
        │   └── sampleB_rat.csv  
        └── bed/  
            └── sampleB_rat.bed
```

publishDir path; mode; pattern; saveAs;

- `saveAs` is a closure:

- in: name of the file captured in the output section (`filename`)

↓ some logic

- out: altered name for publishing

```
.
├─ work/
│   ├── sampleA_mice.fasta
│   ├── sampleA_mice.csv
│   ├── sampleA_mice.bed
│   ├── sampleB_rat.fasta
│   ├── sampleB_rat.csv
│   └── sampleB_rat.bed
└─ results/
    └─ fasta/
        ├── sampleI_mice.fasta
        └── sampleII_rat.fasta
```

```
process PROCESS_SAMPLE {

    publishDir "results/fasta", pattern: "*.fasta",
    saveAs: { filename ->
        if (filename.contains('sampleA')) return "sampleI_${species}.fasta"
        else if (filename.contains('sampleB')) return "sampleII_${species}.fasta"
        else return filename
    }

    input:
    tuple val(sample_id), val(species)

    output:
    path "*.fasta"
    path "*.csv"
    path "*.bed"

    script:
    """
    echo "here a FASTA file" > ${sample_id}_${species}.fasta
    echo "here a CSV file" > ${sample_id}_${species}.csv
    echo "here a BED file" > ${sample_id}_${species}.bed
    """

}
```

publishDir path; mode; pattern; saveAs;

- `saveAs` is a closure:

- in: name of the file captured in the output section (`filename`)

↓ some logic

- out: altered name for publishing

```
.
├─ work/
│   ├── sampleA_mice.fasta
│   ├── sampleA_mice.csv
│   ├── sampleA_mice.bed
│   ├── sampleB_rat.fasta
│   ├── sampleB_rat.csv
│   └── sampleB_rat.bed
└─ results/
    └─ fasta/
        ├── sampleI_mice.fasta -> /abs/path/to/work/sampleA_mice.fasta
        └── sampleII_rat.fasta -> /abs/path/to/work/sampleB_rat.fasta
```

```
process PROCESS_SAMPLE {

    publishDir "results/fasta", pattern: "*.fasta",
    saveAs: { filename ->
        if (filename.contains('sampleA')) return "sampleI_${species}.fasta"
        else if (filename.contains('sampleB')) return "sampleII_${species}.fasta"
        else return filename
    }

    input:
```

```
sample_id), val(species)
```

```
a"
```

```
FASTA file" > ${sample_id}_${species}.fasta
CSV file" > ${sample_id}_${species}.csv
BED file" > ${sample_id}_${species}.bed
```

```
}
```

publishDir path; mode; pattern; saveAs;

- `saveAs` is a closure:

- in: name of the file captured in the output section (`filename`)

↓ some logic

- out: altered name for publishing

```
.
├── work/
│   ├── sampleA_mice.fasta
│   ├── sampleA_mice.csv
│   ├── sampleA_mice.bed
│   ├── sampleB_rat.fasta
│   ├── sampleB_rat.csv
│   └── sampleB_rat.bed
└── results/
    ├── fasta/
    │   ├── sampleI_mice.fasta
    │   └── sampleII_rat.fasta
    └── bed/
        └── Wednesday/
            ├── sampleA_mice.bed
            └── sampleB_rat.csv
```

```
process PROCESS_SAMPLE {

    publishDir "results/fasta", pattern: "*.fasta",
    saveAs: { filename ->
        if (filename.contains('sampleA')) return "sampleI_${species}.fasta"
        else if (filename.contains('sampleB')) return "sampleII_${species}.fasta"
        else return filename
    }

    publishDir "results/bed", pattern: "*.bed",
    saveAs: { filename ->
        def week_day = new Date().format('EEEE')
        "${week_day}/${filename}"
    }

    input:
    tuple val(sample_id), val(species)

    output:
    path "*.fasta"
    path "*.csv"
    path "*.bed"

    script:
    """
    echo "here a FASTA file" > ${sample_id}_${species}.fasta
    echo "here a CSV file" > ${sample_id}_${species}.csv
    echo "here a BED file" > ${sample_id}_${species}.bed
    """
}
```

Summary

- Only files that **match the output declaration** are published.
- `publishDir` can be used **multiple times** to target different files and directories.
- **Process input values** can be used to define the path and/or name of published files.
- Use `pattern` to select which files are published
- Use `saveAs` to rename files or modify their published path
- Use `mode` to change the default symlink
- `publishDir` can be specified in the configuration file